

DSC 40B Discussion 4 Worksheet Solutions

Friday, April 24, 2026

Imagine you are a tutor, and you need to explain something to someone who has only a vague understanding of what's going on. Do your best to get the point across. **ALTERNATE**.

If you're not sure, ask your peer if they know how to explain it, then REPEAT the explanation back. Remember, the point of the exercise is to **vocalize** your thoughts.

If you are both unsure, make sure to come to/tutor for help! Do not use ChatGPT or any other LLM! The point of discussion is to prepare you for the exams.

Have one person in the pair open the Discussion 1 Gradescope assignment. After both you **and** your partner finish a problem **and explain it**, answer the corresponding multiple choice/short answer questions for both versions.. You will receive instant feedback on the questions, whether you are correct or not. If your answer is incorrect, please go back to the question to understand it and feel free to ask tutors about anything you are confused about. Once you have finished the entire worksheet or discussion is ending, turn in your assignment and add your partner to the submission.

Quick Icebreaker

Introduce yourself to your partner by sharing name, year, major, etc. How did you spend your weekend? (Do not spend more than 2 minutes!)

Questions

Question 1:

First Person:

Which of the following is NOT a valid loop invariant for insertion sort on arr? Explain your reasoning.

- A. arr[0, ..., i-1] is sorted
- B. arr[0, ..., i-1] contains the same elements as originally
- C. arr[i, ..., n-1] is unsorted
- D. All elements left of i are $\leq$ all elements right of i

D; Not necessarily true for insertion sort as insertion sort only guarantees that the first k elements are sorted in the correct order. Makes no assumptions about the unseen numbers.

Second Person:

Which of the following is NOT a valid loop invariant for selection sort on arr at the start of iteration i?
Explaining your reasoning.

- A. The subarray arr[0, ..., i-1] is sorted
- B. The subarray arr[0, ..., i-1] contains the i-th smallest elements in the array
- C. The minimum element of arr[i, ..., n-1] will be placed at position i by the end of this iteration
- D. The subarray arr[i, ..., n-1] may be in arbitrary order

C; C is a statement about the elements that have not been processed yet. A loop invariant must be a condition that is true about the current state of the list and/or processed elements.

Question 2:

Second Person:

State and solve the recurrence relation describing the function's run time. Explain your reasoning and show your work.

```
def fool(n):
    if n <= 1:
        return 1

    for i in range(n):
        for j in range(n):
            print('foo')

    return fool(n - 2)
```

$$T(n) = T(n - 2) + n^2$$

$$T(0, 1) = 1$$

$$T(n) = T(n - 2) + n^2 = T(n - 4) + (n - 2)^2 + n^2 = T(n - 4) + (n - 4)^2 + (n - 2)^2 + n^2$$

Can be bounded by $\frac{1}{8}n^3$ and $n^3 \rightarrow \Theta(n^3)$

First Person:

State and solve the recurrence relation describing the function's run time. Explain your reasoning and show your work.

```
def foo2(n):
    if n <= 0:
        return 1

    x = foo2(n - 1)
    y = foo2(n - 1)
    return x + y
```

$$T(n) = 2T(n - 1) + 1$$

$$T(0) = 1$$

$$T(n) = 2T(n - 1) + 1 = 2(2T(n - 2) + 1) + 1 = 4T(n - 2) + 3 = 4(2T(n - 3) + 1) + 3 = \dots$$

Therefore there are 2^n calls of $T(0)$, which results in a $\Theta(2^n)$ runtime.

Question 3:

First Person:

Suppose that we start with an arbitrary list, then the worst case asymptotic runtime of merge sort is faster than that of insertion sort. Explain your reasoning.

☒ True

☐ False

Worst case runtime of merge sort is $\Theta(n \log n)$, while the worst case runtime of insertion sort is $\Theta(n^2)$.

Second Person:

Suppose that we start with an already sorted list, then the asymptotic runtime of merge sort is faster than that of insertion sort. Explain your reasoning.

☐ True

☒ False

Runtime of merge sort on a sorted list is still $\Theta(n \log n)$, while the runtime of insertion sort is $\Theta(n)$.

Question 4:

Second Person:

Suppose that we start with an arbitrary list and perform a modified merge sort, where we split the list into three parts of roughly equal length, recursively sort each part, and then merge the 3 parts. Which of the following describes a recurrence relation for the runtime of this algorithm? Explain your reasoning and show your work.

☒ ~~$T(n) = 3T(n/3) + n$~~

☐ $T(n) = T(n/3) + n$

☐ $T(n) = 3T(n) + n$

☐ $T(n) = 3T(n/3) + n^3$

☐ $T(n) = T(n/3) + n^3$

☐ $T(n) = 3T(n) + n^3$

Each of the three recursive calls would be on $1/3$ of the list, hence $3T(n/3)$. Each "layer" will have to be merged back together, hence the $+ n$.

First Person:

Solve the recurrence relation above. What is the asymptotic time complexity of this modified merge sort? Discuss how it compares to the asymptotic time complexity of standard merge sort. Explain your reasoning and show your work.

- ☐ $T(n) = \Theta(n)$
- ☐ $T(n) = \Theta(n^2)$
- ☐ $T(n) = \Theta(n^3)$
- ☒ ~~$T(n) = \Theta(n \log(n))$~~
- ☐ $T(n) = \Theta(n^2 \log(n))$
- ☐ $T(n) = \Theta(n^3 \log(n))$

$$T(n) = 3T(n/3) + n = 3(3T(n/9) + n/3) + n = 9T(n/9) + 3(n/3) + n = 3^k T(n/3^k) + kn = 3 \log(n) T(n/(3 \log n)) \rightarrow \Theta(n \log n)$$

Question 5:

Example tracing merge sort on the following list: [4, 2, 1, 3]

[4, 2, 1, 3]

[4, 2] [1, 3]

[4] [2] [1] [3]

[2, 4] [1, 3]

[1, 2, 3, 4]

First Person:

Referring to the example above and implementation in lecture, trace out running merge sort on the following list: [4, 9, 1, 25, 8, 7, 15, 5].

How many calls of `mergesort()` and `merge()`?

6 calls of `mergesort()` (7 including the original call on the whole list)

7 calls of `merge()`

Second Person:

Referring to the example above and implementation in lecture, trace out running merge sort on the following list: [10, 4, 17, 2, 9, 1, 13, 6, 8].

How many calls of `mergesort()` and `merge()`?

7 calls of mergesort() (8 including the original call on the whole list)
8 calls of merge()

Question 6:

Second Person:

Suppose we are computing the k-th order statistic of a list of numbers by pivoting on the **middle** element of the list. For which of the following is quickselect expected to perform the **worst**? Explain your reasoning.

- ☐ Sorted list
- ☐ Partially sorted list
- ☐ List sorted in reverse order
- ☒ Arbitrary list

Pivoting on the middle element does not guarantee a balanced split unless the data is randomly distributed, so performance depends on the arrangement of the list. An arbitrary list can produce highly unbalanced partitions, leading to the worst expected behavior. Sorted (or reversely sorted) lists guarantee balanced partitions when pivoting on the middle element.

First Person:

Suppose we are computing the k-th order statistic of a list of numbers by pivoting on the **first** element of the list. For which of the following is quickselect expected to perform the **best**?

- ☐ Sorted list
- ☐ Partially sorted list
- ☐ List sorted in reverse order
- ☒ Arbitrary list

Pivoting on the first element for a sorted (or reversely sorted) list will result in an extremely unbalanced partition, as all other elements are either larger or smaller than the pivot. Therefore in this case, an arbitrary list is most likely to give us the best partitions.

Question 7:

Example tracing of `quickselect()` using Approach #1 in lecture to find the 3rd largest element in [3, 1, 4, 2]:

```
qss(3, 0, 4)
    [3, 1, 4, 2] → [1, 2, 4, 3]
qss(3, 2, 4)
    [1, 2, 4, 3] → [1, 2, 3, 4]
```

→ Return 3 (as pivot is in the target position)

First Person:

Refer to the implementation of `quickselect()` in lecture. Instead of selecting a random index as the pivot index, select the largest index possible. Trace the calls of `quickselect()` to find the 5th largest element in the `[4, 9, 1, 25, 8, 7, 5]`. Use Approach #1 in lecture.

Explanation

```
qss(arr, 5, 0, 7): [4, 9, 1, 25, 8, 7, 5] → [4, 1, 5, 7, 8, 25, 9]
```

```
qss(arr, 5, 3, 7): [4, 1, 5, 7, 8, 25, 9] → [4, 1, 5, 7, 8, 9, 25]
```

```
qss(arr, 5, 3, 5): [4, 1, 5, 7, 8, 9, 25] → [4, 1, 5, 7, 8, 9, 25]
```

→ Return 8

Second Person:

Refer to the implementation of `quickselect()` in lecture. Instead of selecting a random index as the pivot index, select the largest index possible. Trace the calls of `quickselect()` to find the 5th largest element in the `[17, 2, 9, 1, 13, 6, 8]`. Use Approach #1 in lecture.

Explanation

```
qss(arr, 5, 0, 7): [17, 2, 9, 1, 13, 6, 8] → [2, 1, 6, 8, 13, 9, 17]
```

```
qss(arr, 5, 4, 7): [2, 1, 6, 8, 13, 9, 17] → [2, 1, 6, 8, 13, 9, 17]
```

```
qss(arr, 5, 4, 6): [2, 1, 6, 8, 13, 9, 17] → [2, 1, 6, 8, 9, 13, 17]
```

→ Return 9